Moogsoft Generic REST API Integration with AWS Lambda

Date: April 9, 2026 | **Platform:** AWS Lambda + Moogsoft AIOps

---

1. Project Overview

Designed and built a production-ready AWS Lambda function to forward CloudWatch alarm events to Moogsoft's Generic REST LAM API. The solution enables automated incident creation in Moogsoft when AWS infrastructure alarms fire, with auto-resolution when alarms return to OK state.

Business Problem

- CloudWatch alarms were only sending email notifications via SNS — no integration with the enterprise AIOps platform (Moogsoft)
- Manual alert correlation and incident creation was slow and error-prone
- No automated clear/resolve when alarms recovered

Solution

- Generic, reusable Lambda that accepts any CloudWatch alarm format and POSTs to Moogsoft REST API
- Second Lambda (alarm analysis) performs baseline analysis and forwards enriched data to Moogsoft via direct async invoke
- Auto-severity calculation, deduplication via signatures, and auto-clear on OK state

2. Architecture

```
CloudWatch Alarm
    |
    v
Alarm Analysis Lambda (alarm_analysis_lambda.py)
    +-- Fetches 14 days of CloudWatch metric history
    +-- Computes baseline: avg, max, p95, p99
    +-- Compares current value vs baseline + threshold
    +-- SQS enrichment: message age, send/delete rates
    +-- Renders verdict: "UNUSUAL SPIKE" or "LIKELY NORMAL BURST"
    |
    +-- Async invoke (fire-and-forget)
            |
            v
    Moogsoft Forwarder Lambda (app.py)
            +-- Transforms payload to Moogsoft format
            +-- Retry with exponential backoff + jitter
            +-- SSL bypass for corporate certs
```

```
            |
            +-- POST /generic_rest
                    |
                    v
              Moogsoft AIOps (Alert created / resolved)
```

3. Technical Design — Moogsoft Forwarder Lambda

OOP Architecture and Design Patterns

Pattern	Class	Purpose
Strategy	`AuthStrategy` hierarchy	Pluggable authentication (NoAuth, Bearer, Basic, Header, HMAC)
Factory	`PayloadTransformer` hierarchy	Payload transformation (Passthrough, Envelope, MoogsoftNative, CloudWatchAlarm)
Composite	`CompositeSecretProvider`	Chain local env to AWS Secrets Manager
Singleton	Module-level `_FORWARDER`	Reused across warm Lambda invocations

Key Classes

Class	Responsibility
`AppConfig`	Frozen dataclass — all env vars parsed once at init
`BoundedSecretCache`	LRU cache with TTL for secrets (prevents memory leaks)
`RetryHttpClient`	HTTP client with exponential backoff, jitter, SSL bypass
`TargetResolver`	Resolves Moogsoft URL from config or secrets
`MoogsoftForwarder`	Orchestrator — ties auth, transform, retry, and target resolution

Non-Functional Requirements

- **Memory safety:** Soak tested 500 iterations — only 56KB growth (no leaks)
- **Retry:** Exponential backoff with jitter (configurable max retries)
- **SSL:** Corporate self-signed cert bypass via `SSL_VERIFY=false`
- **Idempotency:** Moogsoft signature field enables deduplication
- **Fail-safe:** Moogsoft forwarding failures never crash the analysis Lambda

4. Technical Design — Alarm Analysis Lambda

Capabilities

Feature	Details
Event parsing	EventBridge, direct CloudWatch, and alarmData formats
Baseline computation	14-day history, avg/max/p95/p99 percentiles
SQS enrichment	Message age, send rate, delete rate, producer vs consumer analysis
Verdict engine	"UNUSUAL SPIKE" vs "LIKELY NORMAL BURST" based on percentile comparison
Moogsoft integration	Async direct invoke of forwarder Lambda (Option B)
Auto-clear	OK state sends severity=0 to resolve alert in Moogsoft

Severity Auto-Calculation

Condition	Moogsoft Severity
Spike ≥ 3x threshold	5 — Critical
Spike ≥ 2x threshold	4 — Major
Spike ≥ 1.5x threshold	3 — Minor
Spike < 1.5x threshold	2 — Warning
Alarm returns to OK	0 — Clear

5. Moogsoft REST API Details

Field	Value
Endpoint	`/generic_rest`
Method	POST
Authentication	None (current config)
SSL	--insecure (self-signed corporate certs)
Mandatory fields	source, description, agent, agent_location, class, type, severity
Signature	Dedup + clear matching (source:type:metric_name)
Severity range	0 (Clear) to 5 (Critical)

Sample Payload

```
{
  "source": "my-order-queue",
  "source_id": "123456789012:us-east-1:AWS/SQS/ApproximateNumberOfMessagesVisible",
  "description": "UNUSUAL SPIKE: MyAlarm | current=15000 threshold=5000 p95=3000",
  "agent": "aws-alarm-analysis-lambda",
  "agent_location": "us-east-1",
  "class": "queue",
  "type": "queue_depth_spike",
  "severity": 5,
  "signature": "my-order-queue:queue_depth_spike:ApproximateNumberOfMessagesVisible",
  "additional_data": "{\"baseline_avg\":200,\"baseline_p95\":3000,\"verdict\":\"UNUSUAL SPIKE\"}"
}
```

6. Integration: Direct Lambda Invoke vs SNS Fan-out

Aspect	Direct Invoke (chosen)	SNS Fan-out (alternative)
Complexity	Simple — one invoke call	Requires SNS topic + subscription
Latency	~50ms async invoke	~100ms through SNS
Coupling	Caller knows target function	Decoupled via topic
Fan-out	1:1 only	1:N subscribers
IAM	lambda:InvokeFunction	sns:Publish + subscription

Used `InvocationType="Event"` — async fire-and-forget so the analysis Lambda never waits for Moogsoft.

7. Files and Repository

File	Purpose	Lines
`app.py`	Moogsoft forwarder Lambda (OOP)	~536
`alarm_analysis_lambda.py`	Alarm analysis + Moogsoft integration	~650
`moogsoft_integration.py`	MoogsoftSender class (direct invoke)	~120
`test_app.py`	35 unit tests for forwarder	~400
`soak_test.py`	Memory leak validation (500 iterations)	~60
`send_test.py`	Test payload sender	~150
`echo_server.py`	Local HTTP echo server for testing	~30

Repo: https://github.onefiserv.net/fiserv/moogsoft-generic-rest-lambda

8. Testing Summary

#	Test	Result
1	35 unit tests (pytest)	All passed
2	Soak test — 500 iterations	56KB growth (no leak)
3	Local echo server smoke test	200 OK
4	Live Moogsoft QA test	HTTP 200, processed 1
5	AWS Lambda deployed test	200 OK, 142ms, 57MB

9. Deployment

Moogsoft Forwarder Lambda

- Zip: `moogsoft-forwarder.zip` (5,443 bytes)
- Handler: `app.lambda_handler`
- Runtime: Python 3.12+
- VPC: Required (route to Moogsoft internal network, port 443)
- Env: MOOGSOFT_URL, SSL_VERIFY=false, AUTH_TYPE=none

Alarm Analysis Lambda

- Zip: `alarm-analysis-lambda.zip` (7,719 bytes)
- Handler: `alarm_analysis_lambda.lambda_handler`
- Runtime: Python 3.12+
- Env: MOOGSOFT_ENABLED=true, MOOGSOFT_FUNCTION_NAME=moogsoft-forwarder-qa
- IAM: lambda:InvokeFunction on forwarder ARN

10. Challenges and Solutions

Challenge	Solution
PDF binary-compressed	PyPDF2 to extract text programmatically
Corporate SSL blocking HTTP	Local echo server on port 9999
No AWS CLI locally	Manual deployment steps + scripts
Lambda "Device busy" error	VPC routing — needed route to Moogsoft network
Memory leak concerns	Soak test — 56KB over 500 iterations (bounded LRU)
Secrets in repo	Full grep audit, removed hardcoded URLs, .gitignore

11. Key Technical Decisions

- **OOP over procedural:** Strategy, Factory, Composite patterns for extensibility
- **Direct invoke over SNS:** Simpler, fewer resources, sufficient for 1:1
- **Async fire-and-forget:** Moogsoft never blocks analysis
- **Signature-based dedup:** Same signature on ALARM/OK enables auto-resolve
- **No external dependencies:** Only boto3 (pre-installed) and urllib3 (stdlib)
- **Terraform over CloudFormation:** Team preference

12. Interview Talking Points

- **End-to-end ownership:** Reviewed API docs (PDF), designed architecture, wrote code, tested locally, deployed to AWS, tested live, pushed to repo
- **Production readiness:** Retry with backoff+jitter, bounded caches, soak testing, secret management, SSL handling
- **Design patterns:** Strategy (auth), Factory (transformers), Composite (secrets) — justified by real extensibility needs
- **Integration patterns:** Evaluated SNS fan-out vs direct invoke, chose direct invoke for simplicity
- **Security:** Audited for secrets before git push, SSL bypass only where needed, IAM least-privilege
- **Troubleshooting:** VPC routing, corporate SSL, binary PDF extraction — adapted each time
- **Testing strategy:** Unit tests (35), soak tests (memory), local echo server, live QA validation

Generated April 9, 2026 | Moogsoft Generic REST Lambda Integration